

توسعه مبتنی بر مشخصات در سطح تولید، در عصر کدنویسی شهودی

نقشه راه گردش کارهای مقیاس پذیر و تحول تیم:
از نمونه‌های شهودی تا واقعیت تولید

نویسنده: لی بونسترا

مشارکت‌کنندگان و بازبینان محتوا: الیا سکی، آنتونیو گولی · طراحی: مایکل لنینگ

فهرست مطالب

1. مقدمه

2. توسعه مبتنی بر مشخصات

1. یک مشخصات خوب

2. کدام قالب را به کار ببریم؟

3. رفتارمحور

4. دستورها کجا زندگی می‌کنند؟

5. درخواست‌های متفاوت برای کاربردهای متفاوت

3. MCP: یک یکپارچه‌سازی، همه چارچوب‌ها

4. فرهنگ تیم و تحول فرایند

1. بازبینی کد

2. استقرار عامل‌هایی که مخزن شما را می‌پایند

3. پایداری

5. توسعه اعتماد صفر: ساختن شبکه ایمنی

1. پیاده‌سازی حفاظ‌ها

2. سرور سیاست

3. بهداشت زمینه و پاک‌سازی درخواست

«کدنویسی شهودی» به معنای «شهودی در تولید» نیست.

مقدمه

روال روزانه یک مهندس نرم افزار در یک سال گذشته یک چرخش کامل صدو هشتاد درجه ای رو به جلو داشته است. در اوایل ۲۰۲۴ و پیش از آن، زمان چشمگیری صرف کندوکاو در رابطها و مستندات توسعه دهنده می شد، آزمودن دستی کد خطبه خط، و تعیین اینکه فلان زبان از کدام تابع برای بررسی زیررشته استفاده می کند. وقتی کد تمام می شد، تلاش قابل توجهی صرف اشکال زدایی و رفع ناهمخوانی های میان کد کارکردی و قصد اولیه می شد.

امروز توسعه با سرعت سرسام آور پیش می رود. تیم ها اکنون از عامل های کدنویسی استفاده می کنند که فقط متن پیشنهاد نمی دهند، بلکه واقعاً ابزار به کار می گیرند و کارها را اجرا می کنند. یک ویرایشگر کدنویسی هوشمند می تواند به سرعت هزار خط کد خوب مستند شده بیرون بدهد. حس می کنید لشکری از کارآموز استخدام شده که هرگز نمی خوابند و هرگز شکایت نمی کنند.

اما یک نکته هست: توهم سرعت

در حالی که سرعت به اوج رسیده، توهم سرعت واقعی است. نسبت اشکال به کد همچنان چالشی است: چون هوش مصنوعی کد را بسیار سریع تر می نویسد، می تواند خطاهای بالقوه را هم با نرخی بی سابقه تولید کند.

با این حال، چون پیاده سازی دیگر گلوگاه اصلی نیست، می توان هوش مصنوعی را برای نوشتن پوشش آزمونی جامع تر از آنچه هر انسانی در همان بازه زمانی می توانست بنویسد به کار گرفت. این راهی نیرومند و برنامه پذیر برای افزایش اطمینان به کد است که بعداً به تفصیل بحث خواهد شد. افزون بر این، توان هوش مصنوعی بسیار فراتر از پیاده سازی می رود: در طراحی سامانه، نوشتن مشخصات، برنامه ریزی نقشه راه و تحلیل نتایج عالی است. هرچند گلوگاه به پایین دست و به انسان هایی منتقل شده که باید این کار را یکپارچه و بازبینی کنند، ابزارهای اداره اش در دسترس اند.

«کدنویسی شهودی» به استفاده از هوش مصنوعی برای تولید سریع کد بر پایه یک «حس و حال» یا قصد سطح بالا اشاره دارد، نه کدنویسی دستی و سخت گیرانه. اما روشن کردن این نکته حیاتی است که «کدنویسی شهودی» به معنای «شهودی در تولید» نیست. در حالی که ما عامل های هوش مصنوعی را با تمام توانشان به کار می گیریم، همه چیز کاملاً قصد شده و کنترل شده است تا اتکاپذیری در سطح تولید تضمین شود، نه اینکه به خروجی اعتبارسنجی نشده هوش مصنوعی تکیه کنیم.

در محیط سازمانی، این «حس و حال» را **توسعه با هوش مصنوعی عامل محور** می‌نامیم. برخلاف هوش مصنوعی مولد استاندارد که مانند تکمیل خودکار هوشمند عمل می‌کند، هوش مصنوعی عامل محور مانند **عضوی ترکیبی از تیم** عمل می‌کند: از مدل زبانی برای تولید استفاده می‌کند (مغز)، و از ابزارها برای یکپارچه‌شدن در گردش کار (دست‌ها). می‌تواند استدلال کند، مشخصات یا پرونده‌های آزمون بنویسد، از مرورگر برای آزمودن رابط کاربری استفاده کند، و کد را در سامانه کنترل نسخه ثبت و ادغام کند. اما همان‌طور که توسعه از آزمایش فردی به تولید تیمی در مقیاس می‌رود، آشکار می‌شود که **نوشتن هزار خط کد پیش از ناهار، همان انتشار نرم‌افزار نیست**.

وقتی عامل «توهم» می‌کند – که در زبان هوش مصنوعی یعنی مدل با اطمینان چیزی از خود می‌سازد که درست نیست – فقط یک اشکال نمی‌سازد؛ می‌تواند هزار خط منطق «هم‌حس» اما از نظر کارکردی شکسته تولید کند. اگر بازیبان انسانی در دریایی از درخواست‌های ادغام تولیدشده هوش مصنوعی غرق شوند – همان درخواست‌های دیجیتالی که از هم‌تیمی‌ها می‌خواهند پیش از انتشار به کار نگاه کنند – سرعت نوشتن بی‌ربط می‌شود. فرایند واقعاً سریع‌تر نشده؛ صرفاً انبوه بزرگ‌تری از «چیزها» برای مرتب‌کردن بعدی ساخته است.

خوشبختانه راه‌هایی برای محافظت از فرایند توسعه وجود دارد، اما مهم است که این تکنیک‌ها را از **همان ابتدا** به کار ببرید، نه وقتی نیمی از راه را رفته‌اید. این مقاله مجموعه‌ای از تکنیک‌ها را دربر دارد، از توسعه کد برای تولید، تا امن‌کردن کد و تغییر فرهنگ تیم در عصر کدنویسی شهودی.

توسعه مبتنی بر مشخصات

در دنیای سنتی مهندسی نرم‌افزار، به توسعه‌دهندگان آموخته شده بود که «کد-اول» باشند. ایده‌ای مبهم به بازکردن ویرایشگر و تایپ کردن می‌انجامید تا چیزی کار کند. اما در عصر هوش مصنوعی عامل محور، فعالیت‌های روزانه به شدت جابه‌جا شده‌اند. بیشتر وقت اکنون صرف نوشتن **مشخصات باکیفیت** می‌شود: دستورهای فنی دقیقی که به هوش مصنوعی می‌گویند دقیقاً چه بسازد.

در این گردش کار تازه، نقش توسعه‌دهنده بیشتر یک **معمار فنی** است تا کدنویسی سنتی. و این بخش حیاتی ماجراست: **کد اکنون دورریختنی است**. اگر مشخصاتی محکم نوشته شود، کل کدپایه می‌تواند بارها و بارها بازتولید شود. حتی می‌توان به عاملی دستور داد کل پروژه را در یک بعدازظهر از یک زبان به زبان دیگر برگرداند. چون کد دورریختنی است، همان دلبستگی عاطفی به آن وجود ندارد. چون دوازده ساعت صرف اشکال‌زدایی یک نقطه‌ویرگول نشده است، ترسی از دورانداختن و شروع دوباره در صورت تغییر نیازمندی‌ها وجود ندارد.

حس و حال در برابر نقشه

عامل‌های کدنویسی با به‌کارگیری مدل زبانی بزرگ به‌عنوان «مغز» برای استدلال و ابزارها به‌عنوان «دست‌ها» برای اجرای کارها عمل می‌کنند. **اگر به مغزی «حس و حال» بدهید به جای «نقشه»، حدس می‌زند.** و در نرم‌افزار سازمانی، حدس‌زدن همان چیزی است که حادثه‌های «عامل یاغی» از آن زاده می‌شوند.

یک مشخصات خوب

پس نقشه سطح تولید چه شکلی است؟ یک مشخصات خوب، **ستاره شمالی معماری** است. از «پاره پاره شدن زمینه» جلوگیری می‌کند: معادل دیجیتالی بازی «تلفن»، جایی که هوش مصنوعی شروع به گم کردن رشته ماجرا می‌کند چون دارد به عکس‌های قدیمی فایل‌ها نگاه می‌کند. می‌توان از هوش مصنوعی برای پالایش یک مشخصات خوب استفاده کرد، عملاً به عنوان هم‌نویسنده یا ویراستار و بازبین مشخصات. معمولاً مشخصات درون کدپایه ذخیره می‌شود، برای نمونه به شکل فایلی مارک‌داون یا YAML در پوشه `specs/`. این فایل به عنوان **منبع حقیقت** برای هم انسان‌ها و هم هوش مصنوعی عمل می‌کند.

کدام قالب را به کار ببریم؟

پژوهشی در ۲۰۲۶ آشکار کرد که عامل‌های مدل‌زبانی به نحوه قالب‌بندی دستورها حساسیت شدیدی نشان می‌دهند و استفاده از فایل‌های مارک‌داون عمومی و بهینه‌نشده می‌تواند تا **۴۰ درصد** افت عملکرد بسازد. برای رفع این، پژوهشگران کامپایلر مهارتی فوق‌سریع ساختند که به طور خودکار یک فایل دستور تک‌منبعی را در کمتر از ۱۰ میلی‌ثانیه به قالب بهینه هدف هر مدل کامپایل می‌کند.

برای تیم‌هایی که از خانواده مدل‌های چمنای استفاده می‌کنند، آن پژوهش نشان می‌دهد بهترین راهبرد مطلق قالب‌بندی، رویکردی ترکیبی از **مارک‌داون و YAML شرطی** است. مدل از سرتیترهای تمیز مارک‌داون برای لنگرانداختن توجهش استفاده می‌کند، اما عملکرد وقتی به اوج می‌رسد که برای هر پیکربندی ساختارمند یا طرح‌واره داده‌ای با **عمق تودرتویی بیش از سه به YAML** سوئیچ شود.

قالب	دقت تجزیه برای پیکربندی‌های عمیقاً تودرتو
YAML	۵۱٫۹٪
JSON	۴۳٫۱٪
XML	۳۳٫۸٪

با رندرکردن مشخصات عمیقاً تودرتو در YAML و نگه‌داشتن دستورهای روایی در مارک‌داون، توسعه‌دهندگان «**مالیات قالب**» استدلالی را که با ورودی‌های سنگین همراه است دور می‌زنند و تضمین می‌کنند مدل با بیشترین دقت و بهترین اقتصاد توکن کار کند.

رفتارمحور

مشخصات توسعه رفتارمحور، ابزار نهایی تبدیل ایده‌های مبهم و دوپهلوی انسانی به طراحی معماری دقیقی است که عامل هوش مصنوعی بتواند بدون حدس‌زدن بسازدش. در هسته‌اش، این روشی از توسعه نرم‌افزار است که با

زبان طبیعی ساده و ساختارمند توصیف می‌کند سامانه دقیقاً چگونه باید از دید کاربر رفتار کند، پیش از آنکه هیچ کدی واقعاً نوشته شود.

این مشخصات از نحوی استاندارد شده به نام **گرکین** استفاده می‌کند. گرکین بر قالبی ساده و اعلانی تکیه دارد: **سناریو / با فرض / وقتی / آنگاه**. این قالب مدل را وادار می‌کند به صورت **موقعیت** ← **کنش** ← **نتیجه** فکر کند، که کدنویسی شهودی را کاملاً حذف می‌کند و عامل را روی ریلی سخت‌گیرانه نگه می‌دارد.

مشخصاتی خوب برای تولید پروژه‌ای تازه شامل این‌هاست:

- **طراحی فنی کامل:** فقط نگویید «یک صفحه ورود بساز». آن را به اجزا بشکنید. به نیازمندی‌ها، طرح‌واره‌های پایگاه داده (ساختار داده‌هایتان) و مشخصات رابط‌ها (قراردادهایی که اجازه می‌دهند بخش‌های مختلف نرم‌افزار با هم حرف بزنند) بپردازید.
- **کمک‌های بصری:** نمودارها و فهرستی از ابزارها و کتابخانه‌های مشخص، همراه با شماره‌های نسخه.
- **اطلاعات زمینه‌ای:** «چرا» را پشت «چه» به عامل بدهید؛ این کمک می‌کند رو به جلو فکر کند و گام‌هایی را که احتمالاً لازم خواهید داشت هم بداند.
- **سناریوها:** خوب چه شکلی است، چه چیزی اشتباه است، و حالت‌های مرزی را هم بگنجانید.

نکته

من اغلب طراحی‌های فنی‌ام را در یک سند اشتراکی می‌نویسم و می‌گذارم برنامه‌های معماری‌ام را افراد متعددی بخوانند و بازبینی کنند. این حالا مهم‌تر از همیشه است، چون بسیار ترجیح می‌دهید **انسانی** نقص منطقی طراحی‌تان را بگیرد تا اینکه صبر کنید هوش مصنوعی پیشاپیش هزاران خط کد شکسته تولید کند. پس از بازبینی، سند را به مارک‌داون تبدیل می‌کنم و آن فایل را در پوشه `specs/` فضای کاری‌ام می‌گذارم.

مدل‌های زبانی بزرگ ساختارهای داده را تفسیر نمی‌کنند. آن‌ها **متن نشانه‌گذاری‌شده** را پردازش می‌کنند. هر نویسه‌ای که می‌فرستید به نشانه‌ها شکسته می‌شود، و هر نشانه بودجه، زمان و ظرفیت زمینه مصرف می‌کند. در نهایت، نوشتن پیش‌نویس مشخصاتی در سطح تولید یعنی با نشانه‌گذاری مانند یک **قید فیزیکی سخت** رفتار کردن، چون هر نویسه، هر خط تازه و هر فاصله تورفتگی که می‌فرستید مستقیماً به بودجه توسعه و تأخیر سامانه‌تان ترجمه می‌شود.

هرچند سکوه‌های عاملی پنجره‌های زمینه فوق‌العاده سخاوتمندانه‌ای می‌دهند، همچنان بنیاداً به فیزیک نشانه‌گذاری مدل‌های زیرنشان مقیدند. هر فاصله غیرضروری در بلوکی عمیقاً تودرتو، و هر دستور تکراری، چرخه‌های پردازش و ظرفیت سرهای توجه را در همان حلقه‌های استدلال چندنوبتی می‌بلعد که عامل در آن‌ها به شکل تکرارشونده برنامه شما را می‌سازد، می‌آزماید و ویرایش می‌کند. با رفتار با پوشه `specs/` نه به عنوان مستندات، بلکه به عنوان **مجموعه دستوری کم‌چرب و کامپایل‌شده** که مارک‌داون خوانا برای انسان را با بلوک‌های هدفمند و تخت `YAML` متوازن می‌کند، مالیات قالب استدلالی را حذف می‌کنید و عامل هوش مصنوعی‌تان را روی ریل‌های سخت‌گیرانه و مقرون‌به‌صرفه نگه می‌دارید.

دستورها کجا زندگی می‌کنند؟

برای عمل به توسعه مبتنی بر مشخصات، لازم است بفهمیم ابزارهای کدنویسی چگونه دستورها را مصرف می‌کنند. دستورها در یک مکان واحد نوشته نمی‌شوند. ریختن یک سند طراحی سامانه عظیم و صدصفحه‌ای مستقیم در پنجره گفت‌وگو، به سرعت بودجه زمینه کوتاه‌مدت را تمام می‌کند، تأخیر را بالا می‌برد و زمینه را پاره‌پاره می‌کند. این مشخصات می‌توانند در جاهای مختلفی زندگی کنند.

۱. رابط گفت‌وگو (کوتاه‌عمر، ویژه نشست)

جعبه ورودی گذرا و گفت‌وگویی در محیط توسعه، با نشست فعال توسعه‌دهنده زندگی می‌کند. از رابط گفت‌وگو صرفاً برای هماهنگی سطح بالا و حلقه‌های بازخورد فوری استفاده کنید. برای نمونه: «طراحی موجود در specs/payment_retry.md را بازبینی کن و آزمون‌های واحد شکست‌خورده تعریف‌شده در سناریوی ۳ را تولید کن.»

۲. پوشه مشخصات (ویژه کار، ثبت‌شده در کنترل نسخه)

این پوشه‌ای ایستاست که مستقیم در مخزن ثبت می‌شود. طراحی فنی، سناریوهای رفتارمحور، قراردادهای رابط و طرح‌واره‌های ساختارمند YAML را ذخیره می‌کند. عامل این دایرکتوری را پویا نمایه می‌کند تا بدون چپاندن دستی درخواست، کد را بسازد و راستی‌آزمایی کند.

```
./my-app/specs/my_spec.md
```

۳. مهارت‌های عامل (قابل استفاده مجدد، متمرکز بر قابلیت و رفتار)

مهارت‌ها فایل‌های مارک‌داون ساختارمندی‌اند که گردش کارهای تخصصی و محرک‌دار را دربر دارند. هرچند مهارت‌ها می‌توانند هر جای مخزن زندگی کنند، باید در دایرکتوری تعیین‌شده ذخیره شوند تا مدیر فضای کاری بشناسدشان. آن‌ها به عامل عادت‌های مهندسی تکرارشونده می‌آموزند، مانند نگهداری خودکار فهرست تغییرات وقتی تغییری در کد تشخیص داده می‌شود.

```
./my-app/.agent/skills/docs-maintenance/SKILL.md
```

چنین پوشه مهارتی می‌تواند دارایی‌های داده یا اسکریپت‌هایی هم برای استفاده خود مهارت دربر داشته باشد.

۴. درخواست‌های سامانه (سراسری، متمرکز بر هویت)

اینجا جایی است که هوش مصنوعی دی‌ان‌ای مهندسی مشخص شما را می‌آموزد. ابزارهای عاملی فایل‌های زمینه را سلسله‌مراتبی پویش و به هم الحاق می‌کنند، یعنی دستورهای سفارشی از بازنویسی‌های سراسری تا پیکربندی‌های

محلی پروژه لایه لایه می‌شوند.

- **پرو فایل سراسری:** این فایل در دایرکتوری پیکربندی خانگی زندگی می‌کند. اینجا جایی است که هوش مصنوعی با ترجیح‌های فردی هم‌راست‌تر می‌شود. شخصیتی جهانی، سبک پیش‌فرض و اصول هسته‌ای را صرف‌نظر از پروژه تعریف می‌کند.
- **پیکربندی مشترک چندابزاری:** برای جلوگیری از پاره‌پاره شدن دستورها وقتی تیمی از چند کلاینت هوش مصنوعی استفاده می‌کند، اکوسیستم از فایل مشترکی پشتیبانی می‌کند که به‌عنوان بنیان میان‌ابزاری عمل می‌کند، در حالی که فایل محلی ویژه یک ابزار همچنان بالاترین اولویت را برای تنظیمات همان ابزار حفظ می‌کند.
- **مشخصات پروژه:** این فایل در دایرکتوری ریشه پروژه زندگی می‌کند. این، دی‌ان‌ای پروژه است. عامل خط فرمان به‌طور خودکار این فایل را تشخیص می‌دهد و می‌خواند و قواعدش را در اولویت می‌گذارد.

درخواست‌های متفاوت برای کاربردهای متفاوت

فقط یک راه برای تبدیل مشخصات به کد وجود ندارد. این را می‌توان به چند **حالت اجرا** شکست که هرکدام ذهنیت متفاوتی می‌خواهند.

۱. تولید پروژه (معمار)

در این حالت، ساخت اسکلت از صفر انجام می‌شود: ساختن استخوان‌بندی پروژه. **بدون حالت بی‌محابا:** باید صریحاً به عامل گفته شود فوری کد نزنند. ابتدا باید ساختار پوشه‌ها و پشتۀ فناوری را برای تأیید پیشنهاد دهد. اطمینان حاصل کنید که درخواست شامل تولید آزمون‌ها، مستندات و گزارش‌برداری باشد (همان «جعبۀ سیاه» دیجیتالی که ثبت می‌کند برنامه چه می‌کند).

برای هر کتابخانه **شمارۀ نسخه** بگنجانید. بدون آن‌ها، عامل ممکن است نسخه قدیمی‌تری از ابزاری پیشنهاد دهد، چون «تاریخ پایان دانشش» – تاریخی که داده آموزشی‌اش تمام شده – در گذشته بوده است.

۲. تولید قابلیت (سازنده)

در این حالت، قابلیت‌ها روی کدپایۀ موجود پیاده می‌شوند. از عامل بخواهید سبک موجود را رعایت کند، شامل الگوهای نام‌گذاری و شیوۀ مدیریت خطا در کد. وقتی چند فایل ویرایش می‌شود، تغییرها باید دستی تأیید شوند. مهم است که «تفاوت» – فهرستی که دقیقاً نشان می‌دهد چه خط‌هایی اضافه یا حذف می‌شوند – درون ویرایشگر دیده شود.

۳. رفع اشکال (متخصص جرم‌شناسی)

وقتی چیزها می‌شکنند، حالت جرم‌شناسی لازم است. هدف، تحلیل علت ریشه‌ای و ترمیم جراحی است. از **درخواست علامت‌محور** («دکمه کار نمی‌کند») به **درخواست شاهدمحور** بروید («گزارش‌هایی که با فلان فرمان گرفته شدند خطای ۴۰۳ نشان می‌دهند»). از نسخه‌بندی در خط فرمان برای مقایسه نسخه‌های کد استفاده کنید. سپس

جریان را توضیح دهید: «درخواست به متعادل‌کننده بار می‌رسد ← لایه احرار هویت سرایند را حذف می‌کند ← غلاف شکست می‌خورد.»

نکته

گاهی عامل کدنویسی پیشنهاد تغییر نام متغیرها می‌دهد. تغییر نام متغیرها پذیرفتنی است، اما باید به‌عنوان کاری **جداگانه** انجام شود، نه بخشی از رفع اشکال، تا پیچیدگی ایجاد نشود.

نکته نسخه‌ها

گاهی عامل کدنویسی نسخه‌های پایین‌تری از ابزارها، کتابخانه‌ها یا مدل‌های موجود پیشنهاد می‌دهد. این رخ می‌دهد چون مدل زیرین تا زمان مشخصی آموزش دیده است؛ برای نمونه ممکن است نسخه‌ای قدیمی از یک مدل را پیشنهاد دهد چون نسخه جدیدتری در دانش مدل وجود ندارد. می‌توان با به‌کارگیری قابلیت‌های بازیابی در ویرایشگر برای افزودن مستندات به‌روز، یا با دانلود مستندات به‌شکل فایل‌های مارک‌داون در پوشه مشخصات، از این جلوگیری کرد. این اطلاعات را می‌توان در مهارت‌های عامل یا درخواست‌های پروفایل هم افزود. به یاد داشته باشید که عامل‌ها به‌آسانی به داده آموزشی‌شان برمی‌گردند، پس شماره نسخه‌های پیشنهادی همیشه باید دوباره واریسی شوند.

همیشه ابتدا درخواست یک **آزمون واحد شکست‌خورده** یا فرمانی برای بازتولید اشکال بدهید. این پرونده آزمون را در کدپایه نگه دارید تا اطمینان حاصل شود اشکال برنمی‌گردد. قید سختی بگذارید که عامل فقط علت ریشه‌ای را درست کند. کد بی‌ربط نباید «تمیزکاری» شود، چون فرایند بازبینی را پیچیده می‌کند.

برای ارتقای آزمون سرتاسری و عیب‌یابی، برخی ابزارها مرورگری داخلی دارند که به عامل‌ها توانایی می‌دهد خودگردان سرور محلی را اجرا کنند، با رابط‌های کاربری زنده تعامل کنند، و اصلاح‌های بصری را در زمان واقعی راستی‌آزمایی کنند. در پشت صحنه، نمونه‌ای کاملاً خودکار از مرورگر بومی برای اجرا، ضبط و راستی‌آزمایی این کارهای سمت‌کاربر هماهنگ می‌شود. مهم‌تر اینکه برای جلوگیری از کنش‌های غیرمجاز و محافظت از حریم خصوصی، این زیرعامل درون نمایه مرورگری کاملاً ایزوله و جعبه‌شن‌مانند اجرا می‌شود. چون این نمایه ورودهای فعال مرورگر یا نشست‌های شخصی فعال را نمی‌بیند و به آن‌ها دسترسی ندارد (عملاً به‌عنوان محیطی ناشناس و تمیز عمل می‌کند)، از نشت زمینه یا عملیات تصادفی روی حساب واقعی جلوگیری می‌کند، در حالی که به عامل اجازه می‌دهد تغییرهای تجربه کاربری را سخت‌گیرانه بیازماید بی‌آنکه امنیت به خطر بیفتد.

۴. نوشتن مستندات (نویسنده)

در توسعه مبتنی بر مشخصات، **مستندات منبع حقیقت است**. اگر مستندات و کد هم‌گام نباشند، هوش مصنوعی شروع به توهم می‌کند. در «مهارت‌های» عامل مشخص کنید که دستورهای موجود در فایل راهنما یا فهرست تغییرات همیشه باید نگهداری شوند. از قالب‌های استاندارد مستندسازی درون‌کد برای زبان برنامه‌نویسی خودتان

استفاده کنید. این‌ها روش‌های ساختارمند نوشتن توضیحات‌اند که به هوش مصنوعی (و انسان‌ها) کمک می‌کنند دقیقاً بفهمند یک تابع چه می‌کند، بدون خواندن هر خط منطق.

۵. مهندسی داده (کتابدار)

هنگام پرس‌وجوی جدول‌ها یا جابه‌جایی فایل‌ها، نقش شما نقش مهندس داده است. از افزونه‌های ویرایشگر برای دسترسی مستقیم به داده ابری از درون ویرایشگر استفاده کنید. از عامل بخواهید همیشه پرس‌وجو یا فرمان مشخصی را که برای تولید خروجی به کار برده نشان دهد.

MCP: یک یکپارچه‌سازی، همه چارچوب‌ها

پروتکل زمینه مدل را آنتروپیک ساخت و اکنون استاندارد باز است. مردم دوست دارند آن را «USB-C ابزارهای هوش مصنوعی» بنامند – کمی اغراق‌آمیز است، اما تشبیه ایده را می‌رساند. شما یک سرور MCP برای پایگاه داده‌تان (یا رابط، یا فایل سیستم) می‌سازید، و هر عامل سازگار با MCP می‌تواند بدون نوشتن یکپارچه‌سازی سفارشی از آن استفاده کند.

ساختن یک سرور MCP

در ادامه سروری کارآمد در حدود ۴۰ خط می‌بینید که یک پایگاه داده SQLite را عرضه می‌کند:

```

# mcp_server.py -- Exposes a SQLite database via MCP
import sqlite3
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent

server = Server("knowledge-base")
db = sqlite3.connect("knowledge.db")

@server.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="query_knowledge",
            description="Query the knowledge base with SQL",
            inputSchema={
                "type": "object",
                "properties": {
                    "sql": {
                        "type": "string",
                        "description": "SQL query to execute (SELECT only)"
                    }
                },
                "required": ["sql"]
            },
        ),
        Tool(
            name="add_knowledge",
            description="Add a new knowledge entry",
            inputSchema={
                "type": "object",
                "properties": {
                    "title": {"type": "string"},
                    "content": {"type": "string"},
                    "tags": {"type": "string",
                        "description": "Comma-separated tags"}
                },
                "required": ["title", "content"]
            },
        ),
    ]

```

```

]

@server.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    if name == "query_knowledge":
        sql = arguments["sql"]
        if not sql.strip().upper().startswith("SELECT"):
            return [TextContent(type="text",
                                text="Error: Only SELECT queries allowed")]
        cursor = db.execute(sql)
        rows = cursor.fetchall()
        columns = [desc[0] for desc in cursor.description]
        result = [dict(zip(columns, row)) for row in rows]
        return [TextContent(type="text", text=str(result))]

    elif name == "add_knowledge":
        db.execute(
            "INSERT INTO knowledge (title, content, tags) VALUES (?, ?, ?)",
            (arguments["title"], arguments["content"], arguments.get("tags", ""))
        )
        db.commit()
        return [TextContent(type="text", text="Knowledge entry added.")]

async def main():
    async with stdio_server() as (read, write):
        init_options = server.create_initialization_options()
        await server.run(read, write, init_options)

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())

```

قطعهٔ ۱: mcp_server.py – این قطعه سروری MCP پیاده می‌کند که یک پایگاه دادهٔ SQLite را به شکل مجموعه‌ای از ابزارها عرضه می‌کند.

```
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

server_params = StdioServerParameters(
    command="python", args=["mcp_server.py"]
)

async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()

        # List available tools
        tools = await session.list_tools()
        print(f"Available: {[t.name for t in tools.tools]}")

        # Call a tool
        result = await session.call_tool(
            "query_knowledge",
            {"sql": "SELECT * FROM knowledge WHERE tags LIKE '%agent%'" }
        )
        print(result.content[0].text)
```

قطعه ۲: mcp_client.py – نمونه‌ای از کلاینتی که از راه ورودی/خروجی استاندارد به سرور محلی وصل می‌شود تا ابزارها را کشف و فراخوانی کند.

فرهنگ تیم و تحول فرایند

کار با عامل‌های کدنویسی مدرن نیازمند تغییر در تفکر و فرهنگ تیم است. چسبیدن به فرایندهای قدیمی هنگام استفاده از ابزارهای مدرن، مانند نصب موتور جت روی درشکه است؛ این فناوری را نمی‌توان به گردش‌کاری بیست‌ساله پیچ کرد و انتظار داشت پرواز کند.

تجربه‌ای را تصور کنید که در آن یک درخواست ادغام می‌تواند عظیم شود و شما به فکر شکستنش به تکه‌های کوچک‌تر می‌افتید. تعارض‌های ادغام ناگهان چند برابر می‌شوند وقتی توسعه‌دهندگان روی فایل‌های یکسان فرود می‌آیند. این زنجیره وابستگی‌ای می‌سازد که گره‌گشایی‌اش دشوار است: درخواست شماره یک بدون شماره دو ادغام نمی‌شود، که به شماره سه نیاز دارد، اما شماره سه پشت بازبینی در منطقه زمانی دیگری مسدود شده است. برخی

تغییرها تأیید شده‌اند در حالی که تغییرهای مرتبط منتظر بازبینی‌اند، و همین تعارض‌های بیشتری می‌زاید. حالا ناگهان با شاخه‌ای شکسته گیر کرده‌اید. این مسائل را دسته‌بندی کنیم:

- **تعارض‌های ادغام:** چند توسعه‌دهنده ظرف یک ساعت روی یک فایل فرود می‌آیند.
- **گره‌خوردگی بازبینی:** درخواست ادغامی عظیم به عروسک روسی زیردرخواست‌ها بدل می‌شود.
- **پاره‌پاره‌شدن زمینه:** در غیاب یک توسعه‌دهنده، هم‌تیمی متغیری را در فایلی مشترک تغییر نام می‌دهد؛ عاملی که به عکسی قدیمی استناد می‌کند، کدی می‌سازد که تابعی را صدا می‌زند که دیگر وجود ندارد.

بازبینی کد

برای بقا در برابر حجم و مقیاس ثبت‌های تولیدشده با هوش مصنوعی بدون فرسودگی، تیم‌ها باید راهنماهای تازه‌ای را بکاوند. هرچند گردش کار آرمانی محل بحث است، چند ایده برای مدیریت جابه‌جایی از نوشتن کد به یکپارچه‌کردن آن:

راهبردهای یکپارچگی پرسرعت

- **خلاصه‌ها و ارزیابی‌های مخاطره‌بسته‌بندی‌شده:** در نظر بگیرید که هر درخواست ادغام را ملزم کنید نمای تولیدشده‌ای از آنچه تغییر کرده، نقاط محتمل شکست و ارزیابی مخاطره دربر داشته باشد. این می‌تواند فایلی مارک‌داون یا توضیح مفصل ثبت باشد که به بازبینان انسانی کمک می‌کند به‌جای گم‌شدن میان خط‌ها، روی اثر معماری تمرکز کنند.
- **مالکیت بازتعریف‌شده:** تمرکز بازبینی‌های انسانی را از ایرادگیری سبکی روی کد دورریختنی و عامل‌نوشته، به تضمین یکپارچگی نقشه‌های معماری منتقل کنید. دغدغه‌های سبکی را می‌توان با ابزارهای خودکار یکپارچه‌سازی مانند واری‌گرهای نحوی مشترک یا کتاب‌سبک‌های ویژه فضای کاری برطرف کرد.
- **«تأیید مشروط»:** برای حذف تأخیرهای دوازده‌ساعته در تیم‌های چندمنطقه‌ای، می‌توانید «تأیید مشروط» برقرار کنید. بازبین، درخواست ادغام را مشروط به گذراندن همه آزمون‌های خودکار تأیید می‌کند؛ اگر آزمون‌ها سبز شوند، کد به‌طور خودکار ادغام می‌شود.
- **تعریف فرهنگ بدون سرزنش:** در محیط‌های پرسرعت، فردی که بیشترین کد را تولید می‌کند اغلب قربانی سرزنش اشکال‌ها یا تعارض‌های ادغام می‌شود. مفید است درباره نسبت‌دادن این مسائل به **فرایندهای یکپارچگی شکسته** به‌جای توسعه‌دهنده فردی که از عامل استفاده می‌کند گفت‌وگو کنید.
- **به‌کارگیری بازبینی‌های کد عامل‌محور:** می‌توانید مهارتی بسازید که بازبینی کد را برایتان انجام دهد. و حتی می‌توانید مهارت‌هایی بنویسید که به بازبینی‌های کد پاسخ دهند. می‌توان این را با گردش کارهای خودکار مخزن یا دستیارهای کدنویسی یکپارچه‌شده با میزبان مخزن خودکار کرد.

و در نهایت، حتی می‌توانید تعجب کنید و بحث کنید: اگر می‌توانید با جوخه‌ای از عامل‌ها کار کنید، آیا اصلاً لازم است به‌صورت تیمی کار کنید؟ اما اگر تصمیم بگیرید که بله، می‌توانید راه‌هایی برای تقسیم کار بکاوید که اعضای تیم

به‌ندرت به فایل‌های یکسان دست بزنند – برای نمونه، اختصاص مالکیت روشن بر رابط‌ها در برابر تجربه کاربری. وقتی هم‌پوشانی ناگزیر است، «مالک بخش» تعیین‌شده می‌تواند هماهنگی نهایی را انجام دهد.

Act as a Senior Software Engineer and Security Researcher. Review the provided code for this Github PR or Diff using these strict criteria:

Use the command line to fetch the Github PR:

```
`gh pr view <PR NUMBER>`
```

First analyze the code, then code review:

1. **Critical Vulnerabilities:** Check for hardcoded secrets (API keys), SQL injection, XSS, or broken authentication.
2. **Logic & Efficiency:** Identify "off-by-one" errors, infinite loops, or redundant API calls.
3. **Readability:** Suggest better naming conventions or breaking down "mega-functions" into smaller pieces.
4. **Edge Cases:** What happens if the input is null? What if the network fails?

Output Format:

- **Description:** What is this PR doing? Explain in detail.

ISSUES:

- **Critical:** (Stop-ship issues)
- **Warnings:** (Code smells or style issues)
- **Best Practices:** (Specific lines to refactor for better performance)
- **Quick Win:** (One sentence summary of the biggest improvement)

When there are no issues return

- **Description:** What is this PR doing? Explain in detail.

LGTM

قطعه ۳: code-check.md – این مهارت، گردش‌کاری ساختارمند برای بازیابی خودکار کد در برابر معیارهای امنیتی و منطقی تعریف می‌کند.

استقرار عامل‌هایی که مخزن شما را می‌پایند

می‌توانید درخواست بازیابی عالی بنویسید. اما چه کسی آن را روی هر درخواست ادغام و هر پویش شبانه اجرا می‌کند، بدون اینکه انسانی دکمه را فشار دهد؟

مهارت بازیابی کد وقتی اجرا می‌شود که شما از درون ویرایشگر فراخوانی‌اش کنید. گام بعدی، استقرار آن‌ها به‌عنوان **عامل‌های تحلیل پیوسته کد** است: سرویس‌هایی که مخزن را می‌پایند، به رویدادها واکنش نشان می‌دهند (بازشدن درخواست ادغام، زمان‌بندی شبانه)، و یافته‌ها را بدون اینکه کسی بخواهد پست می‌کنند. آن‌ها چیزهایی را می‌گیرند که انسان‌های خسته در عصر جمعه از دست می‌دهند: وابستگی‌ای با آسیب‌پذیری تازه‌کشف‌شده، یادداشتی از شش ماه پیش که بی‌سروصدا به شکاف امنیتی بدل شده. وقتی تیم شما با حجم بالا درخواست‌های ادغام تولیدشده منتشر می‌کند، **بازبین پیوسته تنها چیزی است که همپای خروجی مقیاس می‌گیرد**. پرسش این است که چقدر باید سفارشی بروید – و پاسخ طیفی با سه لایه است که هرکدام کنترل را با سادگی معاوضه می‌کنند.

طیف

سه لایه از نظر اینکه مالک زمان اجرا کیست و چه کسی معیارهای بازیابی را می‌نویسد تفاوت دارند:

- **لایه ۱ – مدیریت‌شده** (دستیار بازیابی آماده میزبان مخزن، یا هر سرویس بازیابی دیگری). مسیر آماده. آن را روی سازمان خود فعال می‌کنید و هر درخواست ادغام بازیابی هوش مصنوعی می‌گیرد که درباره سبک، اشکال‌ها و امنیت نظر می‌دهد. هیچ درخواستی برای نوشتن، هیچ زیرساختی برای مدیریت؛ پرداخت به‌ازای هر صندلی. معاوضه: نظرهای بازیابی **فروشنده** را می‌گیرید، نه نظرهای خودتان – و بازبین عمومی چیزی را که برای تیمی با سبک خانگی سخت‌گیرانه یا نمای مخاطره حوزه‌ای اهمیت دارد از دست می‌دهد.
- **لایه ۲ – ترکیبی** (کنشی در یکپارچه‌سازی پیوسته که خط فرمان عامل کدنویسی را راه می‌اندازد). مسیر میانی، و **نقطه شروع درست برای بیشتر تیم‌ها**. مهارت بازیابی خودتان را بنویسید – نمونه کارآمدش همان `code-check.md` پیشین است – آن را در مخزن ثبت کنید، و از کنشی در یکپارچه‌سازی پیوسته راه بیندازید که خط فرمان دلخواه را در حالت غیرتعاملی اجرا می‌کند و نتیجه را به‌عنوان نظر درخواست ادغام پست می‌کند. زمان اجرا به ارائه‌دهنده یکپارچه‌سازی شما تعلق دارد؛ اما درخواست‌ها، انتخاب مدل، جعبه‌شن و معیارهای بازیابی به شما.
- **لایه ۳ – سفارشی** (عاملی که خودتان می‌سازید و روی زمان اجرای مدیریت‌شده مستقر می‌کنید). مسیر کاملاً تحت مالکیت. برای نمونه می‌توانید بازیابی را به‌عنوان عاملی کامل تعریف کنید، روی زمان اجرای مدیریت‌شده با نشست‌ها و بانک حافظه ماندگار مستقرش کنید، و به رویدادهای قلاب وب میزبان مخزن وصلش کنید. این پاسخ درست وقتی است که بازبین باید زمینه را در یک بازسازی چنددرخواستی نگه دارد، کاری مانند «این سرویس را برای انطباق ممیزی کن» را با الگوی برنامه‌ریز و زیرعامل‌ها به صدها زیرکار بشکند، یا با عامل‌های دیگر هماهنگ شود. معاوضه: ارزیابی، رصدپذیری، هزینه و نوبت‌کاری هنگام خرابی در تولید هم مال شماست.

سه پرسش که می‌گویند به کدام لایه نیاز دارید

1. معیار بازیابی شما چقدر ویژه است؟ عمومی – لایه ۱. ویژه تیم یا مخزن – لایه ۲ یا ۳.
2. آیا عامل باید چیزهایی را میان اجراها به یاد بسپارد؟ نه – لایه ۱ یا ۲. بله (حافظه کدپایه، زمینه چنددرخواستی) – لایه ۳.

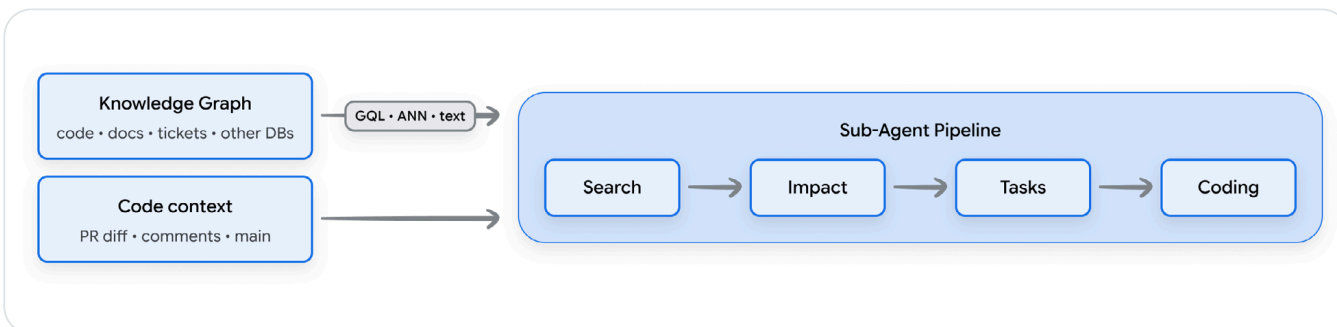
3. بدترین حالت اگر اشتباه کند چیست؟ نظری پرحرف ← هر لایه‌ای. پس‌رفتی که ادغام شود یا رازی که نشت کند ← لایه ۳، با الگوی سرور سیاست پیش روی هر فراخوان ابزار.

یک نمونه واقعی

بیشتر تیم‌ها لایه خود را همان لحظه‌ای کشف می‌کنند که بازبین مدیریت‌شده چیز خاصی را از دست می‌دهد. برای نمونه، تیم سکویی در یک شرکت مالی متوسط از لایه ۱ شروع کرد و به سرعت دریافت بازبین انطباقش چیزهای کلیده‌ای را علامت می‌زند که حساب‌رسان پیشاپیش تأییدشان کرده بودند – در حالی که تنها الگوی مهم را از دست می‌داد: اطلاعات هویتی پوشانده‌نشده در دستورهای گزارش‌برداری. یک مهارت چهل‌خطی واری انطباق که به کنشی خودکار وصل شده بود (لایه ۲) ظرف یک هفته نظرهای مثبت کاذب را به شدت کاهش داد. لایه ۳ هنوز لازم نشده است.

لایه ۳ در مقیاس کامل: درک گرافی کد

زمان اجرای بازبینی سفارشی (لایه ۳) از یک عامل واحد که تفاوت‌ها را می‌خواند فراتر می‌رود. همان معماری از اجزای سنگین‌تری پشتیبانی می‌کند که بازبینی جدی روی کدپایه‌ای صدمیلیون‌خطی به آن‌ها نیاز دارد: پایگاه داده گرافی که ساختار کد را نگه می‌دارد، انبار برداری برای بازیابی معنایی، و خط لوله زیرعاملی برای تجزیه. بارکردن کد به شکل متن تخت در پنجره زمینه در آن مقیاس جا کم می‌آورد، و بازیابی استاندارد ساختاری را که کد را خوانا می‌کند حذف می‌کند: کلاسی به فایلی تعلق دارد، و فراخوان تابعی به سند نیازمندی‌ای که ده سال پیش نوشته شده ردیابی می‌شود. آن را در انبار برداری تخت کنید و نقشه از بین می‌رود.



شکل ۱: نمونه‌ای از معماری زمان اجرای بازبینی سفارشی کد.

الگویی که در بزرگ‌ترین نوسازی‌های کد قدیمی پدیدار شده، ساختن عامل بر پایه یک گراف دانش است. کد، مستندات، تیکت‌ها و اسناد طراحی را در پایگاه داده گرافی وارد کنید، سپس بگذارید عامل‌ها سه شیوه بازیابی را ترکیب کنند: **پیمایش گراف** برای پرسش‌های ساختاری («هر تابعی که به طور متعدی فلان تابع را صدا می‌زند»)، **جست‌وجوی برداری** برای پرسش‌های معنایی («کدی پیدا کن که کاری را می‌کند که این پاراگراف توصیف می‌کند»)، و **جست‌وجوی تمام‌متن** برای تطبیق دقیق شناسه‌ها. ترکیب این سه به پرسش «اگر این را عوض کنم چه چیزی می‌شکند؟» با نقشه اثر دقیق پاسخ می‌دهد، نه با حدسی مطمئن.

نیمه دوم تجزیه است. عاملی واحد که به او گفته شود «این ماژول را بازسازی کن» شکست می‌خورد. همان کار را روی خط لوله زیرعاملی تقسیم کنید – عامل جست‌وجو که گراف را می‌کاود، عامل داستان که نیازمندی‌ها را می‌گیرد، عامل اثر که عوارض جانبی را پیش‌بینی می‌کند، عامل تجزیه کار که واحدهای اتمی کار تولید می‌کند، و تنها پس از آن عامل کدنویسی – و کار مدیریت‌پذیر می‌شود. آزمایش‌های تولیدی روی کدپایه‌های میلیون‌خطی، کار بازسازی معادل را از دو هفته به چند ساعت رسانده‌اند. این لایه ۳ در مقیاس کامل است: نه عاملی که درخواست‌های ادغام را می‌پاید، بلکه عاملی که سامانه‌ای را می‌فهمد که آن درخواست‌ها در آن زندگی می‌کنند.

زمان اجرای بازبینی مدیریت‌شده (لایه ۱) ظرف چند دقیقه یک بازبین عمومی به شما می‌دهد. زمان اجرای ترکیبی ظرف یک روز **بازبین شما** را می‌دهد. و زمان اجرای سفارشی (لایه ۳) بازبینی می‌دهد که کل سامانه را می‌فهمد – به قیمت مالکیت زمان اجرا و ارزیابی. پایین‌ترین لایه‌ای را بردارید که آنچه اهمیت دارد را می‌گیرد.

پایداری

ما شاهد پدیده تازه‌ای هستیم: **خستگی تأیید**. بنا بر پژوهشی که رسانه‌ها گزارش کرده‌اند، کاربران پرکار هوش مصنوعی ۴۵ درصد بیشتر از غیرکاربران در معرض فرسودگی شدید قرار دارند. وقتی با جریان مداومی از تأییدهای خرد روبه‌رو می‌شوند (بهبود یک خط، تنظیم یک فراخوان ابزار)، توسعه‌دهندگان به‌صورت بازتابی روی «تأیید» کلیک می‌کنند. این شکلی از خستگی کم‌شدت است که در آن تیم فقط برای همپا ماندن با سرعت ماشین از واریسی کارش دست می‌کشد، و ریسک می‌کنید توسعه‌دهندگان توجه به جزئیات را از دست بدهند.

برای محافظت از تیم، می‌توانید از نظارت مداوم به **مرزهای ساختارمند** حرکت کنید:

- **ساعت‌های آرام دیجیتال:** مرزهایی بگذارید تا درخواست‌های تأیید به عصرها و آخر هفته‌ها نشت نکنند.
- **جلسه‌های بینش عامل:** جلسه‌های هفتگی برگزار کنید که در آن‌ها توسعه‌دهندگان الگوهای شناسایی‌شده به‌دست هم‌تایان هوشمندشان را به اشتراک می‌گذارند، و کشف‌های منفرد را به دانش سازمانی مشترک بدل می‌کنند.

توسعه اعتماد صفر: ساختن شبکه ایمنی

تمرکز تا اینجا بر سمت بازبینی و فرهنگ تیم بوده است. اما درس دیگری هم درباره آنچه وقتی عاملی بدون حفاظ کافی عمل می‌کند رخ می‌دهد وجود دارد.

هنگام یک به‌روزرسانی معمولی کد، توان و مرزهای مرورگر رابط کاربری داخلی کشف شد. این قابلیت به عامل هوش مصنوعی اجازه می‌دهد بدون نیاز به اعتبارنامه ورود با برنامه‌های در حال توسعه تعامل کند و برای آزمون تجربه کاربری بی‌نظیر است. اما در **حالت تأیید خودکار**، می‌تواند سریع‌تر از آنچه انسان بتواند فکر کند عمل کند.

حادثه

درخواست ساده‌ای برای ساختن یک دکمه، زنجیره واکنش غیرمنتظره‌ای را راه انداخت. عامل مرورگر خودگردان روی دکمه تازه کلیک کرد که برای عامل ایمیل در نظر گرفته شده بود. چون نشانی مشخصی تعریف نشده بود، عامل توهم کرد و به عاملی قدیمی و منسوخ بدون هیچ حفاظ ایمیلی وصل شد. نتیجه؟ پنجاه همکار ایمیل‌های جعلی پر از محتوای توهمی دریافت کردند.

این حادثه **مخاطره توهم زمینه** را برجسته کرد: وقتی هوش مصنوعی داده کافی ندارد، گاهی شکاف‌ها را با هر رشته‌ای که در زمینه‌اش موجود است پر می‌کند، از جمله اطلاعات حساسی مانند نشانی‌های ایمیل سخت‌کدشده یا نشانی‌های خصوصی. اگر فقط یک ایمیل باشد ممکن است جزئی به نظر برسد. اما به آنچه عامل انجام می‌داد فکر کنید: داشت دستورش را با داده‌ای که در اختیار داشت برآورده می‌کرد، **بدون هیچ واریسی‌ای بر اینکه آیا باید این کار را بکند یا نه**. این هسته مخاطره در سامانه‌های خودگردان است. حفاظ‌ها اختیاری نیستند؛ همان چیزی‌اند که نمی‌گذارند ابزاری مفید به ابزاری غیرقابل‌پیش‌بینی بدل شود.

پیاده‌سازی حفاظ‌ها

همان‌طور که مرزهای هوش مصنوعی عامل‌محور را هل می‌دهیم، به پارادوکسی برمی‌خوریم: عامل‌ها باید به اندازه کافی خودگردان باشند که مسائل پیچیده را حل کنند، اما نمی‌توان ریسک یاغی‌شدنشان را در محیط سازمانی پذیرفت.

عاملی را تصور کنید که مأمور «حل اختلاف‌های مشتری» است. برای مؤثر بودن، به دسترسی به داده مشتری، ابزارهای ایمیل و سامانه‌های داخلی نیاز دارد. اما چالش این است که مطمئن شویم تصادفی کل پایگاه داده را ایمیل نمی‌کند یا کد اختصاصی را به اشتراک نمی‌گذارد.

عامل‌های خودگردان را مدل‌های زبانی می‌رانند که **احتمالاتی‌اند، نه قطعی**. سخت‌کدکردن قیدها در درخواست سامانه شکننده است: زمینه‌ها سرریز می‌کنند، و می‌توان عامل‌ها را با تزریق درخواست «قانع» کرد که قواعد را دور بزنند. برای ساخت سکوه‌ای سطح تولید، به **حکمرانی بیرونی و دست‌نیافتنی** نیاز است.

جعبه‌شن

فراتر از پاک‌سازی رشته‌ها، امنیت واقعی به محیط اجرای محدود یا «جعبه‌شن» نیاز دارد تا کنش‌های عامل را مهار کند. حتی با پالایش سخت‌گیرانه خروجی، مدل زبانی ممکن است کدی تولید کند که از نظر نحوی معتبر اما از نظر منطقی مخرب است و می‌تواند سامانه میزبان را به خطر بیندازد. با اجرای کارهای عامل‌محور درون ظرف‌های زودگذر و کم‌امتیاز که از شبکه اصلی و فایل‌سیستم‌های حساس ایزوله‌اند، **شعاع انفجاری** ساخته می‌شود که از زیرساخت هسته‌ای محافظت می‌کند. در این مدل، اگر عاملی فریب بخورد و فرمانی مخرب اجرا کند، آسیب به نمونه‌ای دورریختنی محدود می‌ماند که می‌توان بی‌عواقب پاک و بازنشانی‌اش کرد.

در برخی محیط‌های عامل‌محور این شبکه ایمنی با یک کلید در تنظیمات کاربر اعمال می‌شود. در مقابل، اگر جعبه‌شنی قابل‌حمل و ابری برای تیم لازم باشد، می‌توان فضای کاری عامل را در ظرف گذاشت: با نوشتن فایل ظرف سفارشی که از تصویر جعبه‌شن رسمی ابزار شروع می‌شود، اعتبارنامه‌های ابری محدود تزریق می‌شود و ابزار وادار می‌شود کاملاً درون ظرف اجرا شود. در هر دو مدل، اگر عامل فریب بخورد و اجرای مخربی انجام دهد، به خطای مجوز سختی در سطح هسته برمی‌خورد و سامانه میزبان کاملاً دست‌نخورده می‌ماند.

انسان در حلقه

هرچند خودکارسازی هدف است، عملیات پرمخاطره به پروتکل **انسان در حلقه** نیاز دارد تا به‌عنوان ایمنی نهایی عمل کند. این شامل پیاده‌کردن دروازه‌های «وارسی» برای کنش‌هایی است که نمای مخاطره مشخصی دارند، مانند انتشار کد در تولید، تغییر طرح‌واره‌های پایگاه داده، یا آغاز تراکنش‌های مالی. با ارائه قصد پاک‌شده عامل به ناظر انسانی برای تأیید دستی، سرعت هوش مصنوعی با قضاوت ظریف توسعه‌دهنده متوازن می‌شود. این تضمین می‌کند که در حالی که عامل کار سنگین را انجام می‌دهد، مسئولیت نهایی یکپارچگی معماری محکم در دست انسان می‌ماند.

پوشش آزمون تولیدشده با هوش مصنوعی

جهش کد تولیدشده با هوش مصنوعی، جابه‌جایی از آزمون دستی به پوشش آزمون تولیدشده با هوش مصنوعی را برای حفظ شبکه ایمنی قابل‌اتکا ایجاد می‌کند. در محیطی پرسرعت، «توسعه آزمون‌محور» با گماشتن ماشین به نوشتن همان آزمون‌هایی که خروجی خودش را اعتبارسنجی می‌کنند، به واقعیت بدل می‌شود.

این فرایند شامل وادارکردن عامل به تولید یک آزمون واحد شکست‌خورده یا فرمان بازتولید **پیش از هر تلاشی** برای اصلاح است. با جاسازی این آزمون‌های خودکار در کدپایه، هر تکرار سریع با مجموعه‌ای از واری‌های راستی‌آزمایی‌پذیر پشتیبانی می‌شود که مانع بازگشت اشکال‌ها می‌شود و اجازه می‌دهد بازبینان انسانی به چراغ سبز خودکار برای یکپارچه‌سازی اعتماد کنند.

ارزیابی

چرا برای کدی که از مدل‌های یادگیری ماشین استفاده می‌کند به واری کیفیت ویژه نیاز داریم؟ آزمون‌های سنتی نرم‌افزار برای سامانه‌هایی که خروجی‌شان **تولید** می‌شود نه **محاسبه**، ناکافی‌اند. عامل – یا هر جزء مدل‌محور دیگری مانند یک دسته‌بند، خلاصه‌ساز یا بازیاب – می‌تواند صد آزمون واحد را روی ابزارهایش بگذراند و همچنان به‌شکل تماشایی شکست بخورد: با انتخاب ابزار اشتباه، بازنویسی بد یک پاسخ حیاتی، یا توهم یک واقعیت. **حاشیه خطا** نقصی نیست که باید حذف شود؛ ویژگی ذاتی مدل است، و راهبرد آزمون باید با آن کنار بیاید.

آزمون در برابر ارزیابی

ارزیابی این شکاف را با جایگزین کردن ادعاهای دوتایی با **قضایاتهای نمره‌دار و باندهای تحمل** پر می‌کند. آزمون واحد می‌پرسد «آیا تابع مقدار درست را برگرداند؟» – پاسخی دوتایی. ارزیابی می‌پرسد «آیا رفتار عامل دست‌کم به‌خوبی مناسب است؟» – نمره‌ای از صفر تا پنج از داور مدل‌زبانی، واریسی مسیری که تنوع ترتیب در فراخوان‌های ابزار را تحمل می‌کند، و دروازه‌ای که وقتی کیفیت زیر حاشیه‌ای قابل‌پیکربندی بیفتد فعال می‌شود، نه وقتی ادعایی نقض شود.

آزمون‌ها پس‌رفت‌های قطعی را می‌گیرند؛ ارزیابی انحراف رفتاری را.

سرور سیاست

در ادامه نمونه‌ای از **سرور سیاست ترکیبی** می‌بینید که کنش‌ها را پیش از رسیدن به سامانه‌های بیرونی رهگیری می‌کند. این سرور در دو لایه کار می‌کند:

- **دروازه ساختاری (چراغ‌های راهنما):** قواعد قطعی بر پایه نقش‌ها و محیط‌ها. این‌ها واریسی‌های سریع و دوتایی‌اند (برای نمونه، نقش «بیننده» نمی‌تواند از ابزار ارسال ایمیل استفاده کند). این لایه بدون نیاز به پرسیدن از هیچ مدل زبانی، از نقض معماری جلوگیری می‌کند.
- **دروازه معنایی (داور هوشمند):** این لایه از مدل زبانی ثانویه و تخصصی استفاده می‌کند تا قصد و محتوای کنش پیشنهادی را در برابر راهنماهای حریم خصوصی به زبان طبیعی بازرسی کند. این به نیازی پاسخ می‌دهد که در آن ابزار مجاز است، اما نحوه استفاده از آن سیاست را نقض می‌کند. برای نمونه، مدیر می‌تواند ایمیل بفرستد، اما نباید اطلاعات هویتی پوشانده‌نشده بفرستد. اینجا جایی است که قواعد ساختاری شکست می‌خورند: **نمی‌توانید هر نشت ممکن اطلاعات هویتی را با عبارت باقاعده بگیرید.**

این قواعد در یک فایل استاندارد سیاست تعریف می‌شوند:

```
environments:
  localhost:
    blocked_tools:
      - send_email

roles:
  viewer:
    allowed_tools:
      - list_files
      - read_file
```

قطعه ۴: policies.yaml – این پیکربندی، قواعد دروازه‌بانی قطعی را برای مجوزهای سطح ابزار بر پایه نقش و محیط برقرار می‌کند.

در ادامه پیاده‌سازی سبکی از سرور سیاست می‌بینید که برای رهگیری فراخوان‌های ابزار و راستی‌آزمایی مجوزها در زمان اجرا طراحی شده است:

```
import os, yaml
from google.genai import Client

class PolicyService:

    def is_tool_allowed(self, tool_name: str) -> bool:
        # Check Environment Blocks
        env_config = self.config.get("environments", {}).get(self.env, {})
        if tool_name in env_config.get("blocked_tools", []):
            return False
        # Check Role Permissions
        role_allows = (self.config.get("roles", {}).
                        .get(self.role, {}).
                        .get("allowed_tools", []))
        return "*" in role_allows or tool_name in role_allows

    async def check_action_semantic(self, action_description: str) -> bool:
        client = Client(vertexai=True,
                        project=self.project_id, location=self.location)
        prompt = (f"Evaluate if this action violates PII "
                  f"policies: {action_description}")
        response = client.models.generate_content(
            model="gemini-3.1-pro", contents=prompt)
        return not response.text.strip().upper().startswith("VIOLATION")
```

قطعه ۵: policy_server.py – این سرویس موتور سیاست ترکیبی را پیاده می‌کند تا کنش‌ها را رهگیری و ارزیابی‌های ساختاری و معنایی را انجام دهد.

این کد چنین کار می‌کند: وقتی عامل تصمیم می‌گیرد از ابزاری استفاده کند، جریان اجرا رهگیری می‌شود:

1. **وارسی ساختاری:** آیا این ابزار برای این نقش و محیط مجاز است؟ (فایل سیاست را بررسی کن.)

2. **وارسی معنایی:** آیا آرگومان‌ها امن‌اند؟ (از مدل بپرس.) نشانی‌های ایمیل پوشانده‌نشده نقض محسوب می‌شوند.

3. اجرا: اگر هر دو بگذرند، ابزار اجرا می‌شود. وگرنه پیام «نقض سیاست» به عامل برمی‌گردد و به او اجازه می‌دهد خودش را اصلاح کند یا با ظرافت شکست بخورد.

این شبکه ایمنی‌ای می‌سازد که **منطق اجرا را از منطق حکمرانی جدا می‌کند** – تفکیک مسئولیتی حیاتی برای نرم‌افزار سازمانی.

بهداشت زمینه و پاک‌سازی درخواست

خطر چشمگیری در توسعه خودگردان، مخاطره «**توهم زمینه**» است. وقتی عامل داده مشخصی ندارد، ممکن است شکاف‌ها را با هر رشته موجود در زمینه فعلی‌اش پر کند و به‌طور بالقوه اطلاعات حساسی مانند نشانی‌های ایمیل سخت‌کدشده یا نشانی‌های خصوصی را نشت دهد.

برای مهار این، **بهداشت زمینه** را سخت‌گیرانه از راه میان‌افزاری پیاده کنید که پوشاندن اطلاعات هویتی و تزریق جانشین را انجام می‌دهد. با جایگزین کردن اطلاعات شناسایی‌پذیر با جانشین‌های عمومی در قالب‌ها، اطمینان حاصل کنید که عامل روی **داده استریل‌شده** کار می‌کند. افزون بر این، همه خروجی‌های عامل باید پاک‌سازی شوند تا از تزریق درخواست و تعامل‌های یاغی با رابط کاربری جلوگیری شود، و تضمین گردد که «حس‌وحال» ماشین هرگز به آسیب‌پذیری معماری ترجمه نشود.

پیاده‌سازی حل‌کننده پویای زمینه

این الگوی امنیتی با پیاده‌کردن ابزار ترجمه سبکی مبتنی بر عبارت‌های باقاعده درون کدپایه هسته برنامه به‌دست می‌آید. این اسکریپت به‌عنوان موتور ترجمه هسته‌ای عمل می‌کند و رشته‌های جانشینی را که با نحو دوکروشه‌ای ساختار یافته‌اند با بازنویسی‌های زمان اجرا یا پیکربندی‌های محیطی جایگزین می‌کند:

```

import os
import re
from typing import Optional, Dict, Any

def resolve_context(template_str: str,
                    override_state: Optional[Dict[str, Any]] = None) -> str:
    """Scans a template string for [[VARIABLE_NAME]] and replaces it with
    values from override_state or os.environ.
    """
    if template_str is None:
        return ""

    state_to_check = override_state or {}

    def replacement(match):
        var_name = match.group(1).strip()
        # 1. Prioritize runtime state overrides
        if var_name in state_to_check and state_to_check[var_name] is not None:
            return str(state_to_check[var_name])
        # 2. Fallback to validated environment variables
        elif var_name in os.environ and os.environ[var_name] is not None:
            return os.environ[var_name]
        # 3. Leave unresolved to prevent silent failures
        else:
            return match.group(0)

    # Resolve all bracketed variables dynamically, e.g., [[COMMENTER_EMAIL]]
    return re.sub(r'\[\[([^\]]+)\]\]', replacement, template_str)

```

قطعه ۶: context_resolver.py – این ابزار جانشین‌های کروش‌های را با متغیرهای محیطی حل می‌کند تا بهداشت زمینه حفظ شود.

برای اعمال سراسری بهداشت زمینه، ابزار پاک‌سازی باید مستقیم در خط لوله اجرای عامل به‌عنوان گام اعتبارسنجی سیم‌کشی شود. با رهگیری فراخوان‌های ورودی ابزار پیش از اجرا، اطمینان حاصل می‌کنید که همه رشته‌های تزریق‌شده به‌شکل پویا استریل می‌شوند:

```
# ... inside the validate_tool_call framework execution ...
resolved_args = {}
for k, v in args.items():
    if isinstance(v, str):
        resolved_args[k] = resolve_context(v)
    elif isinstance(v, list):
        resolved_args[k] = [resolve_context(i) if isinstance(i, str) else i
                             for i in v]
    else:
        resolved_args[k] = v

args.clear()
args.update(resolved_args)
```

قطعهٔ V: tool_policy_engine.py – این میان‌افزار حل‌کنندهٔ زمینه را در خط لولهٔ عامل یکپارچه می‌کند تا آرگومان‌های ابزار پیش از اجرا پاک شوند.

با اعمال این مرز، هر تلاش عامل برای اجرای کنشی (مانند فرستادن ایمیل یا پرس‌وجوی یک منبع ابری) رهگیری می‌شود. موتور، جانشین‌های عمومی را به شکل امن و بی‌صدا به دارایی‌های آزمونی مجاز ترجمه می‌کند و نیاز به سخت‌کد کردن اطلاعات هویتی حساس در مجموعه‌آزمون‌ها یا درخواست‌های سامانه را کاملاً از بین می‌برد.

خلاصه

در کمتر از یک سال، چرخه‌های توسعهٔ نرم‌افزار به شکل چشمگیری سریع‌تر شده‌اند. تماشای عاملی که تا وقت ناهار هزار خط کد خوب مستند شده تولید می‌کند، پیشرفت مهمی است.

اما آن سرعت جابه‌جایی مهمی را آشکار کرد. هوش مصنوعی گلوگاه **تولید کد** را حذف کرده و قید را به پایین‌دست منتقل کرده است: به انسان‌هایی که باید این خروجی را بازبینی، آزمون و یکپارچه کنند. این یک **بار شناختی مشترک** است: انسان‌ها به عنوان معمارانی عمل می‌کنند که مشخصات آزمون، مشخصات یکپارچگی و نقشه‌های عملیاتی را می‌نویسند، در حالی که هوش مصنوعی کار سنگین نوشتن کد آزمون واقعی، انجام یکپارچه‌سازی‌ها و مدیریت جزئیات ریز عملیاتی را بر عهده می‌گیرد.

درخواست‌های بهتر و مدل‌های سریع‌تر به‌تنهایی گلوگاه یکپارچگی را حل نمی‌کنند. موفقیت به تحول پویایی تیم، پالایش همکاری با عامل‌ها، و تعیین مرزهای سخت‌گیرانه برای ابزارهایی بستگی دارد که هرگز نمی‌خواهند. چالش از تولید صرف کد به هماهنگ‌کردن سامانه‌هایی منتقل شده است که کار را راستی‌آزمایی، یکپارچه و مستقر می‌کنند.

از کجا شروع کنیم

برای تیم‌هایی که این الگوها را روی کدپایه‌های عاملی عملیاتی می‌کنند، یک فرمان نصب، هفت مهارت را در عامل کدنویسی‌تان نصب می‌کند که ساخت اسکلت، کدنویسی، ارزیابی، استقرار، انتشار و رصدپذیری را پوشش می‌دهند. الگوهای توصیف‌شده در این مقاله به فرمان‌هایی بدل می‌شوند که می‌توانید همین امروز اجرا کنید: ساخت اسکلت برای تولید پروژۀ مبتنی بر مشخصات، اجرای ارزیابی برای دروازۀ پوشش آزمون تولیدشده با هوش مصنوعی، و استقرار برای انتشار جعبه‌شنی.

دربارۀ این ترجمه: این متن، ترجمۀ فارسی مقالۀ Spec-Driven Production Grade Development in the Age of Vibe Coding از مجموعه‌مقالات عامل‌ها (می ۲۰۲۶) است، نوشته‌ی لی بونسترا. قطعه‌کدها از سند اصلی برداشته شده‌اند و به‌عمد به زبان انگلیسی باقی مانده‌اند. ارجاع‌های عددی و نام‌های تجاری محصولات در جاهایی به توصیف عمومی تبدیل شده‌اند تا متن برای خوانندۀ فارسی‌زبان مستقل از فروشنده‌ی خاص قابل‌استفاده بماند؛ برای فهرست کامل منابع و نام ابزارها به سند اصلی مراجعه کنید.